

Maintenance And Troubleshooting

Date: 2026-03-24

Routine Maintenance

Recommended maintenance tasks for the current prototype:

- review backend dependencies in `backend/requirements.txt`
- review frontend dependencies in `frontend/package.json`
- run migrations after any model changes
- reload fixture data only when you intentionally want the seeded activities back in the database
- check `VITE_API_BASE_URL`, `CORS`, and `CSRF` settings if the frontend origin changes
- verify role checks after any authentication or routing changes

Test And Verification

Backend automated tests are in `backend/message_board/tests/`.

Run the backend suite:

```
cd backend
pip install pytest pytest-django
pytest
```

Run a single backend test file:

```
cd backend
pytest message_board/tests/test_summary_service.py
```

Useful frontend verification:

```
cd frontend
npm install
npm run lint
npm run build
```

Common Maintenance Tasks

Apply Database Migrations

```
cd backend  
python manage.py makemigrations  
python manage.py migrate
```

Reload Seeded Activities

```
cd backend  
python manage.py loaddata message_board/fixtures/activities.json
```

Run The Django Development Server

```
cd backend  
python manage.py runserver
```

Run The Frontend Development Server

```
cd frontend  
npm install  
npm run dev
```

Start The Agent Timer Process

```
cd backend  
python manage.py agent_timer
```

This command periodically runs room-state agent checks for active rooms.

Reset Rooms And Non-Superuser Users

```
cd backend  
python manage.py reset_rooms_users
```

Non-interactive form:

```
cd backend  
python manage.py reset_rooms_users --yes
```

Use this carefully. It is destructive.

Troubleshooting

Backend Does Not Connect To The Database

Checks:

- confirm `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_NAME`, and `DB_PASSWORD`
- confirm PostgreSQL is running if you intend to use PostgreSQL
- confirm the configured port matches the database service
- confirm `DB_SSLMODE` is appropriate for the environment

Important implementation detail:

- if `DB_HOST` is missing, Django falls back to SQLite instead of PostgreSQL

Frontend Cannot Reach The Backend

Checks:

- confirm the backend server is running
- confirm `VITE_API_BASE_URL` is set correctly
- confirm the frontend origin is allowed by CORS settings
- confirm the frontend origin is included in `CSRF_TRUSTED_ORIGINS` for authenticated browser requests

Login Or Role-Based Navigation Fails

Checks:

- confirm the user has a related `UserProfile`
- confirm `UserProfile.role` is one of the values supported by the current application code
- confirm the frontend route is protected with the expected role
- confirm you are not trying to use a maintainer frontend route that does not exist in the shipped app

Session Summaries Are Missing

Checks:

- confirm the session produced posts tied to the current `activity_run_id`
- confirm summary generation endpoints were called

- confirm `SessionSummary` rows exist in the database

PDF Export Fails

Checks:

- confirm `reportlab` is installed in the backend environment
- confirm the summary exists before export
- confirm the summary JSON fields are populated with the expected structure

Automated Interventions Are Not Appearing

Checks:

- confirm the room has `activity_is_running=True`
- confirm relevant agents are active
- confirm the selected activity allows the intended agents
- confirm the agent timer process is running if you are testing room-state rules outside direct post submission

Known Risks And Gaps

- no monitoring or alerting stack is included
- no automated backup workflow is included
- destructive maintenance commands exist and rely on operator care
- documentation reflects the shipped prototype, which is narrower than a full production handover